Schema registry

The schema registry provides **repositories of schemas** shared by multiple applications. Previously, without a schema registry, teams might depend on informal agreements—such as

- verbal understandings,
- shared documents not programmatically enforced, or
- wiki pages

to define the message format and how to serialize and deserialize messages. The schema registry ensures consistent message encoding and decoding.

Understand schemas

Imagine you're building an application that tracks customer orders. You might use a Kafka topic called `customer_orders` to transmit messages containing information about each order. A typical order message might include the following fields of information:

- Order ID
- Customer name
- Product name
- Quantity
- Price

To ensure these messages are structured consistently, you can define a schema. Here's an example in Apache Avro format:

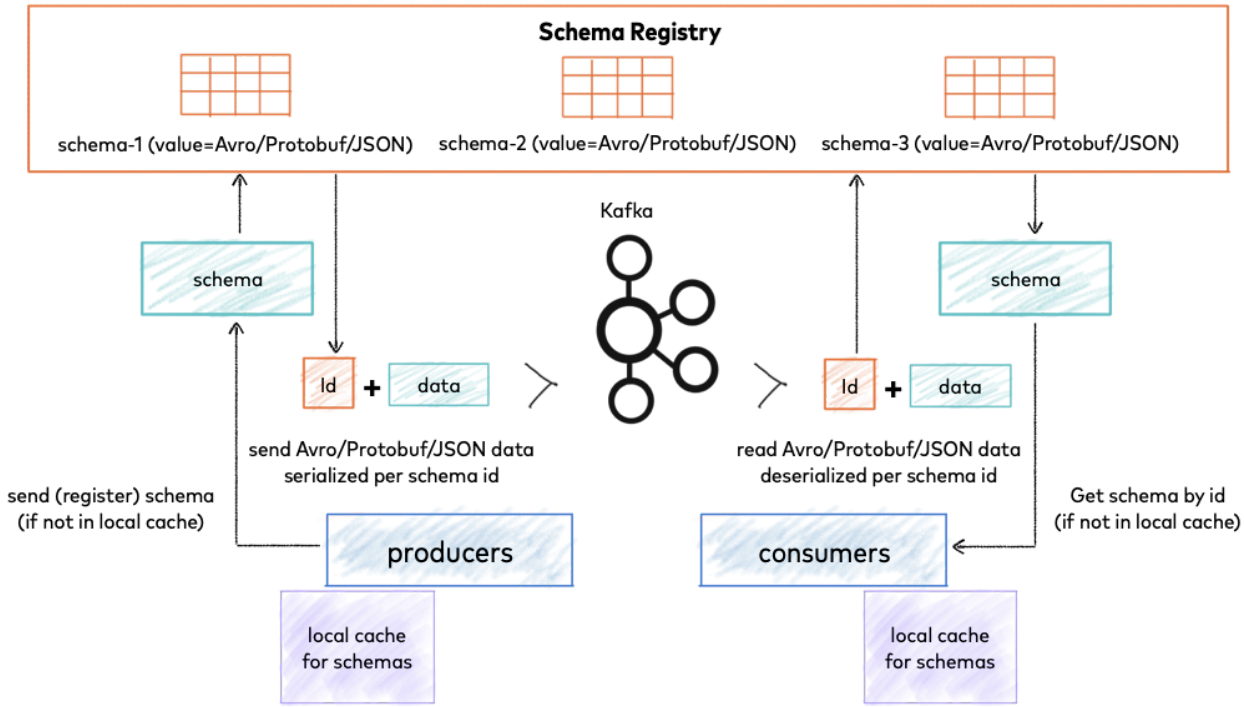
```
{
  "type": "record",
  "name": "Order",
  "namespace": "com.example",
  "fields": [
    {"name": "orderId", "type": "string"},
    {"name": "customerName", "type": "string"},
    {"name": "productName", "type": "string"},
    {"name": "quantity", "type": "int"},
    {"name": "price", "type": "double"}
  ]
```

```
}
```

This schema is a blueprint for a message. It tells you that an `order` message has five fields: an order ID, a customer name, a product name, a quantity, and a price. It also specifies the data type of each field.

If a consumer client receives a binary message encoded with this schema, it must know exactly how to interpret it. To make this possible, the producer stores the `order` schema in a schema registry and passes an identifier of the schema along with the message. As long as a consumer of the message knows which registry was used, it can retrieve the same schema and decode the message.

Schema registry workflow



How Schema Registry Works

When integrated with Kafka clients, Schema Registry follows a specific workflow:

1. **Producer Registration** : Before sending data, a producer checks if its schema is already registered in Schema Registry. If not, it registers the schema and receives a unique schema ID.
2. **Message Serialization** : The producer serializes the data according to the schema and embeds the schema ID (not the entire schema) in the message payload.
3. **Message Transmission** : The serialized data with the schema ID is sent to Kafka.
4. **Consumer Deserialization** : When a consumer receives a message, it extracts the schema ID from the payload, fetches the corresponding schema from Schema Registry, and uses it to deserialize the data.
5. **Schema Caching** : Both producers and consumers cache schemas locally to minimize Schema Registry calls, only contacting it when encountering new schema IDs.

Schema lifecycle management

As your applications and their data requirements change, the structure of your Kafka messages also needs to adapt. Effective schema lifecycle management is crucial for handling these changes smoothly and maintaining data integrity. This process involves not just changing schemas, but also systematically controlling the kinds of changes that are safe, or sufficiently compatible, for the applications that depend on them.

The following available types determine how the schema registry compares a new schema version against previous ones:

- `None`**:** No compatibility checks are performed. Allows any change, but carries a high risk of breaking clients.
- `Backward` **(Default):** Consumer applications using the new schema can decode data produced with only the previously registered schema. Consumers must be upgraded before producers.
- `Backward_transitive`**:** Consumer applications using the new schema can decode data produced with *all* previous schema versions in that subject (subject is the kafka topic appended with “-value”)
- `Forward`**:** Data produced using the new schema must be readable by clients using the previous registered schema.
- `Forward_transitive`**:** Data produced using the new schema must be readable by clients using *all* previous schema versions.
- `Full`**:** The new schema is both backwards- and forwards-compatible with the previously registered schema version. Clients can be upgraded in any order relative to the producer using the new schema.

- **`Full_transitive`**: The new schema is both backwards- and forwards-compatible with all previous schema versions in that subject.

Compatibility Mode	Description	Best For
Backward	New schema can read data written with old schema	When consumers upgrade before producers
Forward	Old schema can read data written with new schema	When producers upgrade before consumers
Full	Both backward and forward compatible	Maximum flexibility

`Example use case`

Assume that you want to add a field, `orderDate`, to your `order` messages. You would create a new version of the `order` schema and register it under the same subject. This lets you track the evolution of your schema over time. Adding a field, to the producer side, is a forwards-compatible change. This means consumers using older schema versions (without `orderDate`) can still read and process messages produced with the new schema, i.e., the new field is ignored. In this example, the initial `order` schema would be version 1. When you add the `orderDate` field, you create version 2 of the `order` schema. Both versions are stored under the same subject, letting you manage schema updates without breaking existing applications that might still rely on version 1.

Compatibility Mode Best practices

- Don't use `None` as a compatibility type strategy because you run the risk of breaking clients with schema changes.
- Choose a forward-based strategy such as `Forward` or `Forward-transitive` if you want to update producers first. Choose a backward-based strategy such as `Backward` or `Backward-transitive` if you want to update consumers first.

- Choose a transitive strategy if you want to maintain compatibility with multiple previous schema versions. If you want to maximize compatibility and minimize the risk of breaking clients when updating schema versions, use the `Full-transitive` strategy.

Reference

1. https://docs.cloud.google.com/managed-service-for-apache-kafka/docs/schema-registry/schema-lifecycle